

Received 31 October 2025, accepted 5 December 2025, date of publication 9 December 2025,
date of current version 19 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3642207

RESEARCH ARTICLE

AdaBoost-LSTM: A Hybrid Ensemble Framework for Sequential Credit Card Fraud Detection

HAMZA H. SULIMANI¹

Department of Computer and Network Engineering, College of Computing, Umm Al-Qura University, Makkah 21955, Saudi Arabia

e-mail: hhhsulimani@uqu.edu.sa

ABSTRACT Credit card fraud detection is a critical task in financial systems, requiring effective algorithms to accurately classify transactions as fraudulent or non-fraudulent. This paper proposes a novel approach, combining the AdaBoost algorithm with Long Short-Term Memory (LSTM) networks, called the AdaBoost-LSTM algorithm, to improve the detection of fraud in sequential transaction data. The AdaBoost algorithm is widely known for its ability to combine multiple weak classifiers into a strong one. In this work, we adapt it to sequential learning tasks by leveraging Long Short-Term Memory (LSTM) networks, which effectively capture temporal dependencies in transaction data. In this approach, multiple LSTM models are trained sequentially, with each model focusing on the misclassified samples from the previous one. Sample weights are adjusted at each iteration to emphasize hard-to-classify instances, and transfer learning is employed to initialize each new LSTM with the parameters learned from the previous iteration, thus reducing computational costs and overfitting risks. The proposed method is evaluated on a credit card fraud detection dataset, demonstrating its effectiveness in improving classification accuracy compared to traditional models.

INDEX TERMS Credit card fraud detection, AdaBoost, long short-term memory (LSTM), sequential learning, transfer learning.

I. INTRODUCTION

The proliferation of data across various domains has driven the development of advanced computational methods and algorithms to tackle increasingly complex problems [12]. With the advent of big data, traditional programming and analytical approaches often struggle to manage the scale, heterogeneity, and complexity of modern datasets [14]. Emerging technologies such as machine learning (ML), deep learning (DL), and artificial intelligence (AI) have demonstrated remarkable capabilities in efficiently processing and analyzing large-scale data, thereby enabling transformative innovations across multiple sectors [19]. For instance, in healthcare [24], DL models are revolutionizing diagnostic imaging and personalized treatment planning; in finance [2], AI-driven systems enhance fraud detection and risk management; and in logistics, ML techniques optimize supply chain operations and demand forecasting. These

advancements have been further accelerated by progress in high-performance computing and the widespread adoption of scalable cloud-based infrastructures.

In the financial technology domain, the integration of artificial intelligence (AI) and machine learning (ML) has transformed the delivery of financial services, particularly in fraud detection, risk assessment, and payment processing [10]. Financial institutions increasingly rely on these advanced technologies to secure transactions, enhance customer experience, and minimize operational risks. Among these challenges, credit card fraud detection has attracted considerable research attention due to the surge in online transactions and the rapid expansion of e-commerce platforms, especially in the post-COVID-19 era [15]. The pandemic accelerated digital transformation, resulting in a sharp increase in credit card usage for online purchases, which in turn has fostered the emergence of more sophisticated fraudulent schemes.

Credit card fraud detection is inherently a classification task, where the primary objective is to distinguish legitimate

The associate editor coordinating the review of this manuscript and approving it for publication was Laura Celentano¹.

transactions from fraudulent ones [4]. Traditional rule-based systems have been largely supplanted by machine learning (ML) and deep learning (DL) approaches due to their superior ability to generalize and adapt to evolving fraud patterns [6], [9]. However, designing efficient DL models for this purpose remains challenging, as it requires the optimization of numerous hyperparameters—a process that is often computationally intensive and time-consuming.

To address the challenges of accurately detecting fraudulent credit card transactions in dynamic and sequential datasets, this paper proposes an innovative framework that integrates Long Short-Term Memory (LSTM) networks with the AdaBoost ensemble learning algorithm. In addition, advanced hyperparameter optimization techniques are employed to fine-tune model performance, enhancing both predictive accuracy and computational efficiency. The proposed framework is specifically designed to capture the temporal dependencies and evolving patterns of fraudulent behaviors, providing a scalable, adaptive, and real-time solution for fraud detection.

This study makes the following key contributions:

- Introduction of the AdaBoost-LSTM algorithm: a novel hybrid model that combines the strengths of AdaBoost and LSTM networks, enhancing the detection of fraudulent patterns in sequential credit card transaction data.
- Sequential adaptation of AdaBoost for temporal learning: multiple LSTM classifiers are trained iteratively, with each subsequent model focusing on the samples misclassified by the previous iteration through dynamic sample reweighting, effectively addressing class imbalance and difficult-to-classify cases.
- Incorporation of transfer learning among LSTM classifiers: each LSTM is initialized with parameters learned from the previous model, reducing training time, computational cost, and the risk of overfitting.
- Integration of Adam-based hyperparameter optimization: adaptive learning rate adjustments accelerate convergence and improve model stability during iterative LSTM training.
- Comprehensive empirical evaluation: the proposed framework is validated on a real-world credit card transaction dataset, demonstrating superior classification accuracy, robustness, and efficiency compared to traditional baseline and ensemble methods.
- Practical applicability for real-time fraud detection: the modular design allows for seamless deployment in operational financial systems, ensuring both scalability and adaptability to evolving fraudulent behaviors.

The structure of this paper is organized as follows. Section II presents a review of the relevant literature, covering Fraudulent Credit Card Transaction Detection using AI in the first part, and AdaBoost combined with Deep Learning in the second. Section III provides an overview of AdaBoost learning, followed by a presentation of the Long Short-Term Memory (LSTM) network. Section IV introduces the proposed methodology for a Hybrid Fraud Detection Framework

using a Boosting-Based LSTM Ensemble. Finally, Section V concludes with an experimental study, comparative analyses, and a discussion of the results.

II. RELATED WORKS

This section provides a comprehensive overview of recent research efforts focused on credit card fraud detection using AI-driven methods. First, we examine conventional ML and DL approaches applied to the classification of fraudulent transactions, highlighting their effectiveness, limitations, and strategies for handling class imbalance. Next, we explore the emerging trend of integrating AdaBoost with deep learning models, which aims to combine the strengths of ensemble learning with the representational power of neural networks. These hybrid frameworks represent a promising direction for developing adaptive, high-performance fraud detection systems capable of addressing the dynamic nature of financial fraud.

A. DETECTION OF FRAUDULENT TRANSACTIONS IN CREDIT CARD USING AI

The widespread adoption of digital payments has transformed financial transactions, enabling convenience and efficiency. However, this digital revolution has also heightened the risk of credit card fraud, presenting a critical challenge for financial institutions. To mitigate this issue, researchers have leveraged artificial intelligence (AI) techniques, including traditional machine learning (ML) algorithms, deep learning (DL) architectures, and hybrid models. This section reviews recent advancements in AI-driven fraud detection systems, emphasizing improvements in accuracy, scalability, and robustness. Machine learning has demonstrated considerable promise in detecting fraudulent transactions. Studies such as [16] and [20] have evaluated various ML classifiers, including Random Forest (RF), Support Vector Machines (SVM), and Logistic Regression (LR). Among these, RF consistently achieved superior performance metrics, including accuracy, precision, and recall. Both studies underscored the critical importance of addressing data imbalance—an inherent challenge in fraud detection—using techniques like Bagging and synthetic data generation methods such as SMOTE. Supervised algorithms, such as Decision Trees, k-Nearest Neighbors (KNN), and AdaBoost, have been benchmarked for fraud detection. In parallel, research on cybersecurity in the digital banking sector [23] has demonstrated the practical use of the Random Forest algorithm on the CICIDS 2017 dataset to detect potential threats and strengthen security infrastructures, further highlighting the relevance of ML-based solutions in financial threat mitigation.

Ensemble methods, including Gradient Boosting and XGBoost, were found to outperform standalone classifiers in predictive accuracy and resilience to overfitting [3], [9]. Feature engineering practices, such as one-hot encoding and feature selection, further enhanced model interpretability and robustness [3]. Deep learning has gained prominence due

to its capacity to analyze large-scale, complex datasets and uncover intricate patterns. Neural network topologies, including Long Short-Term Memory (LSTM) and Artificial Neural Networks (ANNs), have been employed to model temporal dependencies in transactional data [22]. Additionally, [11] introduced an optimized deep learning framework combining Recurrent Neural Networks (RNNs) with Bayesian optimization for hyperparameter tuning. This approach delivered high accuracy with reduced computational overhead. Hybrid frameworks that integrate DL models with other techniques have shown remarkable potential. For instance, [21] explored a combination of autoencoders and deep belief networks, effectively addressing concept drift by incorporating feedback mechanisms. Recent advancements highlight the effectiveness of hybrid and domain-specific approaches. For example, [26] proposed a Deep Forest-based model enriched with transaction-specific features like Individual Credibility Degree (ICD) and Group Anomaly Degree (GAD). This approach significantly improved precision and recall by focusing on outlier detection. Similarly, [1] developed hybrid frameworks that combined multiple classifiers with dynamic feedback mechanisms to adapt to changing fraud patterns. Optimization techniques have been increasingly utilized to refine model performance. Studies such as [8] demonstrated the efficacy of ensemble learning to create super-classifiers, outperforming individual ML models. Additionally, [11] employed Bayesian optimization to fine-tune DL architectures, ensuring adaptability to evolving fraud schemes. Despite significant progress, challenges remain in achieving scalability and handling highly imbalanced datasets. Fraudulent transactions often represent a small fraction of the data, leading to difficulties in maintaining high precision without sacrificing recall. Real-time processing requirements further complicate the development of scalable fraud detection models. Future research should focus on hybrid frameworks that integrate ML, DL, and metaheuristic optimization techniques to address these challenges. Such frameworks can balance the trade-off between false positives and false negatives while ensuring adaptability to evolving fraud patterns, as emphasized in [3] and [11]. AdaBoost, given its strength in addressing misclassification challenges, could play a pivotal role in these frameworks. For instance, combining AdaBoost with deep learning architectures or enhancing its sampling mechanisms through metaheuristic optimization techniques could yield models capable of both precision and adaptability. Additionally, integrating advanced feature engineering, real-time processing capabilities, and explainable AI techniques will be crucial to building robust and interpretable fraud detection systems.

B. ADABOOST WITH DEEP LEARNING

AdaBoost, a robust ensemble learning technique, has been widely combined with deep learning architectures to address challenges in classification and prediction tasks. This synergy capitalizes on AdaBoost's ability to iteratively improve weak

learners and the powerful feature extraction capabilities of deep neural networks, leading to significant gains in accuracy, efficiency, and adaptability. To enhance feature extraction and classification, AdaBoost has been integrated with Convolutional Neural Networks (CNNs). For example, a hybrid AdaBoost-CNN model utilizes convolutional layers and transfer learning to improve classification precision, achieving state-of-the-art performance metrics compared to standalone CNNs [30]. For sequence data, combinations of AdaBoost with Recurrent Neural Networks (RNNs), including Long Short-Term Memory (LSTM) networks, have been proposed. These architectures capture temporal dependencies effectively, with AdaBoost iteratively refining model predictions to optimize learning outcomes [29]. To address the challenges posed by imbalanced datasets, AdaBoost-SVM leverages Support Vector Machines (SVMs) as base classifiers within the boosting framework. This technique reduces SVM hyperparameter space during iterative boosting steps, resolving the trade-off between accuracy and diversity, and consistently outperforming neural network-based AdaBoost variants [17]. In graph-structured data, the Adaboosting Graph Attention Recurrent Network (Ada-GARN) integrates graph attention convolution with gated recurrent units. This model enhances spatial-temporal dependency modeling, achieving lower error metrics such as RMSE and MAE, demonstrating its effectiveness in dynamic datasets [29]. AdaBoost has also been combined with Multilayer Perceptrons (MLPs) to form models like MLPBoost. This approach leverages AdaBoost's iterative learning to enhance MLP performance, resulting in improved accuracy and reduced computational costs, particularly in vision-related tasks [7]. Hybrid models such as AdaBoost-CNN have been developed to handle multi-class imbalanced datasets. By employing transfer learning and CNN-based feature extraction, these models demonstrate significant accuracy improvements and reduced training times, validating their applicability across diverse datasets like CIFAR-10 and Fashion-MNIST [25]. In the domain of regression and forecasting, AdaBoost has been coupled with MLPs in applications such as wind speed prediction. Models like AdaBoost-CG-BP-FR-MLP highlight AdaBoost's potential to fine-tune predictions, improving regression accuracy over conventional MLPs [18]. These methodological advancements underscore AdaBoost's versatility in augmenting deep learning architectures, optimizing performance across diverse tasks, and addressing common challenges like imbalanced data, computational efficiency, and prediction reliability.

III. PRELIMINARIES

In this section, we present the fundamental concepts and theoretical foundations essential for understanding our approach. These preliminaries cover a review of the basic algorithms, the neural network architectures involved, and the optimization methodologies employed.

TABLE 1. Comparison of studies on fraud detection.

Study	Methods	Dataset	Key Findings
[16]	ANN, Decision Trees, SVM, Logistic Regression, Random Forest	Kaggle dataset	ANN achieved the highest accuracy (99.92%), followed by Random Forest (99.21%).
[20]	Random Forest, Bagging Classifier, Logistic Regression, etc.	Kaggle dataset (6.36M rows, 10 columns)	Random Forest best with unbalanced data; Bagging Classifier excelled with balanced data.
[22]	ANN, LSTM, Gradient Boosted Trees, Logistic Regression	Dataset of 80M transactions	Deep Learning topologies like LSTM outperformed traditional methods in scalability and precision.
[29]	Deep Forest with ICD and GAD features	Bank's transaction data	Improved precision by 15% and recall by 20% compared to Random Forest.
[3]	XGBoost, Random Forest, and others	Real-world transaction dataset	XGBoost and Random Forest demonstrated a strong balance of false positives and negatives.
[21]	KNN, Random Forest, SVM, Autoencoders, CNN, RBM, DBN	EU, Australian, and German datasets	Deep Learning methods (e.g., CNN, RBM) performed better on imbalanced datasets.
[8]	Ensemble Learning, Random Forest, SVM	Real-world dataset	Ensemble learning methods enhanced performance; identified critical variables for fraud detection.
[1]	SGD, Decision Tree, Random Forest, IBk, J48	Kaggle dataset (Task1 and Task2)	Random Forest had the highest Kappa statistic; SGD performed worst.
[9]	Sliding Window Strategy, Clustering, Classifiers	European credit card fraud dataset	Behavioral pattern extraction and feedback mechanisms improved prediction accuracy.
[11]	ANN, RNN, LSTM with Bayesian Optimization	Credit card fraud dataset	RNN performed best with optimized parameters, achieving better efficiency and computational performance.

A. ADABOOST LEARNING

The Boosting method aims to enhance the performance of a learning algorithm by improving its accuracy and robustness [5]. Through iterative refinement, Boosting transforms a weak learner into a stronger one by sequentially improving its predictions. One key objective of Boosting is to ensure that, at each training iteration, the base classifier concentrates on the most informative or difficult samples [27]. The final prediction is then obtained by aggregating the outputs of all the base classifiers.

AdaBoost [28], one of the most widely used Boosting algorithms, repeatedly applies a base learner over a series of T iterations, where $k = 1, \dots, T$. A central idea of AdaBoost is to maintain a distribution of weights over the training examples. The weight assigned to a training sample i in round k is denoted by $D_k(i)$. Initially, all examples are assigned equal weights; however, after each iteration, the weights of misclassified examples are increased. This mechanism compels the base learner to focus more on the difficult samples in subsequent rounds, thereby improving overall model performance.

1) DESCRIPTION

Let us consider a training dataset composed of m labeled examples, denoted as $\{(x_1, y_1), \dots, (x_m, y_m)\}$, where each $x_i \in X$ represents the feature vector of the i -th observation, and $y_i \in Y = \{-1, +1\}$ denotes its corresponding class label. Initially, each observation is assigned an equal weight,

as shown in Equation (1):

$$D_1(i) = \frac{1}{m}, \quad \text{for } i = 1, \dots, m. \quad (1)$$

The boosting algorithm proceeds in T rounds, sequentially building a set of weak classifiers $\{h_1, \dots, h_T\}$. At each iteration $t = 1, \dots, T$, the following steps are executed:

1) Weak Learner Selection

Identify a hypothesis $h_t : X \rightarrow \{-1, +1\}$ that minimizes the weighted classification error with respect to the current distribution D_t (see Equation (2)). The resulting weighted error is given in Equation (3).

$$h_t = \arg \min_{h \in \mathcal{H}} \sum_{i=1}^m D_t(i) \cdot [y_i \neq h(x_i)], \quad (2)$$

$$\epsilon_t = \sum_{i=1}^m D_t(i) \cdot [y_i \neq h_t(x_i)]. \quad (3)$$

2) Termination Check

If $\epsilon_t \geq 0.5$, the weak learner performs no better than random guessing, and the algorithm halts. Otherwise, h_t is retained.

3) Classifier Weighting

Compute the importance weight α_t of the selected classifier h_t based on its accuracy, as described in Equation (4).

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right). \quad (4)$$

4) Weight Update

Update the weight distribution over the training examples to emphasize those misclassified by h_t , as shown in Equation (5). The normalization factor Z_t is defined in Equation (6).

$$D_{t+1}(i) = \frac{D_t(i) \cdot e^{-\alpha_t y_i h_t(x_i)}}{Z_t}, \quad (5)$$

$$Z_t = 2\sqrt{\epsilon_t(1 - \epsilon_t)}. \quad (6)$$

After completing T iterations (or stopping earlier due to the error threshold), the final hypothesis is constructed as a weighted majority vote of the weak classifiers, as expressed in Equation (7):

$$H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right). \quad (7)$$

B. LONG SHORT-TERM MEMORY

Long Short-Term Memory (LSTM) networks represent a significant advancement in the realm of recurrent neural networks. They were introduced by Hochreiter and Schmidhuber [13] in 1997 to tackle a critical problem known as the vanishing gradient problem, which arises when distant elements and their temporal context are considered. At the heart of LSTM networks are specialized units called LSTM cells, characterized by their unique memory management through three pivotal “gates” – the Forget gate, Input gate, and Output gate. The overall architecture of an LSTM cell is illustrated in Figure 1.

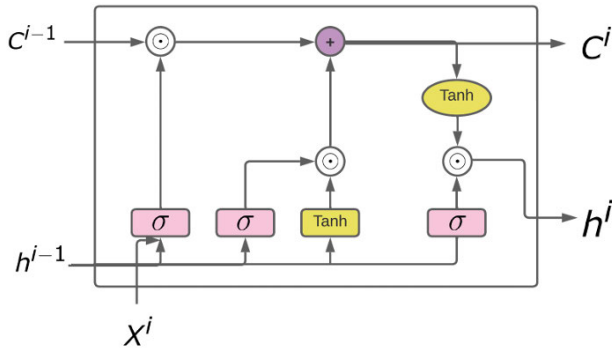


FIGURE 1. Long short-term memory architecture.

In the initial phase of the LSTM system, the forget gate determines which information from the previous cell state should be discarded. This decision is based on two inputs: the hidden state from the previous step, h_{i-1} , and the current input, x_i . The resulting vector, V_i , contains values between 0 and 1 for each element in the previous cell state, C_{i-1} . These values define the extent to which each element of C_{i-1} is retained. The process is mathematically expressed in Equation (8).

$$V_i = \sigma(\Theta_V x_i + \Theta'_V h_{i-1} + b_V) \quad (8)$$

In the subsequent step, the LSTM determines which information from the new input, x_i , should be stored in the updated cell state. This process is represented by Equation (9).

$$I_i = \sigma(\Theta_I x_i + \Theta'_I h_{i-1} + b_I) \quad (9)$$

Next, the input vector, x_i , passes through a block of neurons to produce $\hat{C}_i$, the candidate values for the new cell state. The update to the cell state, C_i , is then calculated using the forget gate, V_i , and the input gate, I_i , as shown in Equations (10) and (11).

$$\hat{C}_i = \tanh(\Theta_C x_i + \Theta'_C h_{i-1} + b_C) \quad (10)$$

$$C_i = V_i \odot C_{i-1} + I_i \odot \hat{C}_i \quad (11)$$

Finally, the hidden state, h_i , is computed using the output gate, O_i , and the updated cell state, C_i , as described in Equation (13). The output gate, O_i , is generated in a manner similar to the forget gate, as defined in Equation (12).

$$O_i = \sigma(\Theta_O x_i + \Theta'_O h_{i-1} + b_O) \quad (12)$$

$$h_i = O_i \odot \tanh(C_i) \quad (13)$$

Here, σ represents the sigmoid activation function, $\odot$ denotes the Hadamard product, and Θ , Θ' , and b are trainable weight matrices and biases.

Adam is a widely used optimization algorithm for adaptive learning rate adjustment, designed to accelerate convergence and enhance model performance [11]. Its functioning is defined by the mathematical equations presented in Equation (14):

$$\begin{aligned} m_k &= \beta_1 \cdot m_{k-1} + (1 - \beta_1) \cdot g_k, \\ v_k &= \beta_2 \cdot v_{k-1} + (1 - \beta_2) \cdot g_k^2, \\ \hat{m}_k &= \frac{m_k}{1 - \beta_1^k}, \\ \hat{v}_k &= \frac{v_k}{1 - \beta_2^k}, \\ \theta_{k+1} &= \theta_k - \alpha \cdot \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}. \end{aligned} \quad (14)$$

In Equation (14), k represents the iteration step, g_k denotes the gradient at step k , and θ_k signifies the parameters being optimized. The terms m_k and v_k correspond to the biased first and second moment estimates, while $\hat{m}_k$ and $\hat{v}_k$ represent their bias-corrected counterparts.

During training, Adam optimizes parameters such as the weights and biases of the LSTM network, which include: $W_{fh}^j, W_{ih}^j, W_{ch}^j, W_{oh}^j, W_{fx}^j, W_{ix}^j, W_{cx}^j, W_{ox}^j$, and biases $b_f^j, b_c^j, b_o^j, b_i^j$. These parameters are essential for accurately modeling temporal dependencies within sequential data.

The pseudocode for the Adam optimization algorithm is presented in Algorithm 2. This algorithm iteratively updates the parameters θ using bias-corrected first and second moment estimates of the gradients, as described in Equation (14).

Algorithm 1 Boosting Algorithm for Classification**Require:** Dataset $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$, with $x_i \in \mathbb{R}^n$, $y_i \in \{0, 1, \dots, C\}$.Maximum number of iterations M , Number of weak classifiers T .

- 1: Initialize weights: $D_1 = (w_{1,1}, \dots, w_{1,N})$ with $w_{1,i} = \frac{1}{N}$.
- 2: **for** $m = 1$ to M **do**
- 3: Train weak classifier $G_m(x)$ using weights D_m .
- 4: Compute error: $e_m = \sum_{i=1}^N w_{m,i} \mathbb{I}(G_m(x_i) \neq y_i)$
- 5: Compute weight: $a_m = \frac{1}{2} \log \frac{1-e_m}{e_m}$
- 6: Update weights:

$$w_{m+1,i} = \frac{w_{m,i} \exp(-a_m y_i G_m(x_i))}{z_m}, \quad z_m = \sum_{i=1}^N w_{m,i} \exp(-a_m y_i G_m(x_i))$$
- 7: **end for**
- 8: Final classifier: $F(x) = \text{sign} \left(\sum_{m=1}^M a_m G_m(x) \right)$

Ensure: Strong classifier $F(x)$.**Algorithm 2** Adam Optimization Algorithm**Require:** Learning rate δ , Objective function L , initial parameters θ , $m_0 = 0$, $v_0 = 0$, $\beta_1, \beta_2 \in [0, 1]$, $k = 0$

- 1: **while** θ not converged **do**
- 2: $k = k + 1$
- 3: $m_k = \beta_1 m_{k-1} + (1 - \beta_1) \frac{\partial L}{\partial \theta_k}$
- 4: $v_k = \beta_2 v_{k-1} + (1 - \beta_2) \left(\frac{\partial L}{\partial \theta_k} \right)^2$
- 5: $\hat{m}_k = \frac{m_k}{1 - \beta_1^k}$
- 6: $\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$
- 7: $\theta_{k+1} = \theta_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}$
- 8: **end while**

Ensure: θ_k **IV. PROPOSED METHODOLOGY**

To address the challenges inherent in credit card fraud detection—such as class imbalance, evolving fraudulent behavior, and the need for real-time decision-making—we propose a hybrid methodology that leverages the strengths of ensemble learning and deep recurrent neural networks. This section introduces the core components of the proposed approach, detailing how boosting strategies and Long Short-Term Memory (LSTM) networks are integrated to form a robust classification framework. The methodology is structured into three key components: (1) an AdaBoost-enhanced LSTM algorithm that incrementally builds an ensemble of temporal classifiers, (2) a hybrid fraud detection framework that organizes the end-to-end learning and detection process, and (3) a practical application of this architecture to credit card fraud detection, encompassing data preprocessing, classification, and feedback mechanisms.

In contrast to conventional AdaBoost formulations that employ shallow decision trees as base learners, the proposed mathematical model redefines the boosting process by embedding LSTM networks as weak learners. This modification preserves the adaptive reweighting mechanism of AdaBoost while integrating the temporal modeling capability of LSTMs (see Algorithm 3). The resulting AdaBoost-LSTM formulation updates sample weights and

ensemble coefficients based on sequential error propagation across LSTM learners—thereby enhancing temporal feature learning and improving the model's capacity to capture complex and evolving fraud patterns. This distinction from traditional approaches clearly demonstrates the originality of the proposed mathematical model, which is further elaborated in the following subsections.

A. ADABOOST-LSTM ALGORITHM

AdaBoost and Long Short-Term Memory (LSTM) networks both employ iterative learning strategies; however, they serve fundamentally different purposes and are grounded in distinct principles. AdaBoost is an ensemble learning technique that constructs a strong classifier by combining multiple weak learners. It iteratively adjusts the weights of training samples, increasing the emphasis on misclassified instances to improve the performance of subsequent learners. This mechanism enables the model to focus on the most informative or challenging samples during training, thereby enhancing overall classification performance.

In contrast, LSTM networks are a type of recurrent neural network (RNN) specifically designed to capture long-term dependencies within sequential data. Through gated memory cells, LSTMs retain and update contextual information over time, making them particularly effective for tasks involving temporal dynamics or ordered inputs. The structure of an LSTM cell is illustrated in Figure 2.

The proposed hybrid model leverages the complementary strengths of both approaches. AdaBoost's sample reweighting mechanism guides an ensemble of LSTM classifiers, effectively aligning the focus of each LSTM on different subsets of the data—particularly those with higher classification difficulty. This integration results in a robust architecture that not only captures temporal dependencies but also dynamically adapts its learning emphasis, thereby improving predictive accuracy.

The AdaBoost-LSTM algorithm iteratively trains an ensemble of LSTM models, each weighted according to its classification performance. The model dynamically updates the weights of training samples, prioritizing those that were

previously misclassified. The final prediction is obtained by aggregating the outputs of all LSTM classifiers, weighted by their respective accuracies.

Regarding convergence, the training of each LSTM model at iteration t is terminated when a stopping criterion is met. This criterion may be defined by the stabilization of the training loss below a predefined threshold, reaching a maximum number of epochs, or the stagnation of a performance metric (e.g., classification error) on a validation set. Furthermore, this stopping rule can be associated with AdaBoost's overall objective via the weighted error threshold ϵ_t . Training stops once any of these conditions are satisfied, ensuring a balance between the local accuracy of each LSTM classifier and computational efficiency.

The training procedure can be summarized as follows:

- 1) Initialize all training sample weights uniformly.
- 2) Train an LSTM model using the current weighted dataset.
- 3) Evaluate the model's performance on the training data.
- 4) Update the sample weights based on prediction errors, increasing the weights of misclassified samples.
- 5) Repeat the process for a predefined number of iterations or until convergence.
- 6) Aggregate the outputs of all trained LSTM models to form the final ensemble predictor.

The complete pseudocode of the AdaBoost-LSTM framework using the Adam optimization algorithm is provided in Algorithm 3. This algorithm iteratively updates the LSTM parameters via Adam (see Equation (14)) while adjusting the sample weights according to AdaBoost's principles, thereby producing a strong classifier through the aggregation of all weak LSTM learners.

B. HYBRID FRAUD DETECTION FRAMEWORK USING BOOSTING-BASED LSTM ENSEMBLE

The proposed architecture introduces a structured and modular pipeline for credit card fraud detection, combining rigorous data preprocessing with a hybrid machine learning model based on boosting and recurrent neural networks. The framework is composed of two main phases: data preprocessing and ensemble model training with evaluation.

In the preprocessing phase, raw transaction data undergoes multiple transformation steps to enhance its quality and usability for modeling. The process begins with data cleaning, where techniques such as noise elimination, outlier detection, and missing data imputation are applied to produce clean and consistent input data. Following this, data reduction is performed through feature selection and extraction to retain the most informative variables, thereby reducing computational complexity. The resulting reduced data is then subjected to data transformation, which includes both standardization and normalization steps to ensure uniform input scales—an essential prerequisite for stable training of LSTM networks.

Once the data is prepared, the machine learning phase begins with data splitting, dividing the dataset into training

and testing subsets. The core of the learning process is a boosting-based ensemble model that integrates two powerful techniques: AdaBoost and LSTM. The AdaBoost module improves predictive performance by iteratively adjusting weights on misclassified instances, enhancing the model's sensitivity to difficult cases and mitigating the impact of class imbalance. Simultaneously, LSTM learning captures temporal dependencies within transaction sequences, a key aspect in modeling the evolving behavior of fraudulent activities over time. These two components work in synergy, forming a hybrid learning system that leverages both temporal awareness and boosted classification.

Finally, the trained model is subjected to evaluation using the reserved test set, where its accuracy, recall, precision, and other performance metrics are measured. This end-to-end architecture ensures not only accurate fraud detection but also adaptability and scalability for real-world financial systems.

C. CREDIT CARD FRAUD DETECTION APPROACH

The proposed methodology combines traditional transaction validation with an advanced hybrid machine learning model to effectively detect fraudulent credit card transactions. Each transaction undergoes a two-level verification process: initial terminal validation (such as PIN or device checks), followed by a machine learning-based classification. If the transaction fails the terminal check, it is immediately rejected. Otherwise, the transaction is encrypted and passed into the fraud detection pipeline. This pipeline begins with feature selection to remove irrelevant or redundant inputs, enhancing the model's efficiency. Next, standardization and normalization are applied to ensure consistent scaling, which is particularly important for the LSTM network. The core of the system is a trained AdaBoost-LSTM model that combines the strengths of two algorithms: AdaBoost, which improves classification accuracy by focusing on previously misclassified instances, and Long Short-Term Memory (LSTM), which captures temporal patterns in sequential transaction data to detect complex fraud behavior. Based on this analysis, the model outputs a classification—either “Fraud” or “Not Fraud.” To maintain model performance over time, a feedback mechanism evaluates the prediction accuracy using real-world outcomes (e.g., user reports of fraud). This feedback can be incorporated into future training cycles to improve the system's adaptability. The architecture offers several key advantages: enhanced robustness through dual-layer validation, temporal awareness via LSTM, improved learning on difficult cases through AdaBoost, and modular scalability that facilitates future upgrades such as swapping feature selection techniques or classifiers.

The overall pipeline is illustrated in Figure 3.

V. IMPLEMENTATION AND RESULT ANALYSIS

A. DATASET

This subsection provides a detailed description of the credit card dataset used for computation (Table 2). The dataset

Algorithm 3 AdaBoost-LSTM Classification Framework With Adam Optimization**Require:** Dataset $T = \{(x_1, y_1), \dots, (x_N, y_N)\}$, where $x_i \in \mathbb{R}^n$, $y_i \in \{0, 1, \dots, C\}$;Maximum boosting iterations M ; LSTM hyperparameters; Adam parameters α , β_1 , β_2 , ϵ

1: Initialize training sample weights: $D_1 = (w_{1,1}, \dots, w_{1,N})$ with $w_{1,i} = \frac{1}{N}$

2: **for** $m = 1$ to M **do**

3: Initialize LSTM parameters $\theta^{(m)}$, Adam variables: $m_0 = 0$, $v_0 = 0$, $k = 0$

4: **while** LSTM not converged **do**

5: $k \leftarrow k + 1$

6: Compute gradient: $g_k = \frac{\partial L^{(m)}}{\partial \theta_k}$ using weighted loss with D_m

7: $m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$

8: $v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2$

9: $\hat{m}_k = \frac{m_k}{1 - \beta_1^k}$

10: $\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$

11: $\theta_{k+1} = \theta_k - \alpha \frac{\hat{m}_k}{\sqrt{\hat{v}_k + \epsilon}}$

12: **end while**

13: Let $G_m(x)$ be the trained LSTM model with parameters θ_k

14: Compute error: $e_m = \sum_{i=1}^N w_{m,i} \cdot \mathbb{I}(G_m(x_i) \neq y_i)$

15: Compute model weight: $a_m = \frac{1}{2} \log \left(\frac{1 - e_m}{e_m} \right)$

16: Update sample weights:

$$w_{m+1,i} = \frac{w_{m,i} \cdot \exp(-a_m y_i G_m(x_i))}{z_m}, \quad z_m = \sum_{i=1}^N w_{m,i} \cdot \exp(-a_m y_i G_m(x_i))$$
17: **end for****Ensure:** Strong classifier:

$$F(x) = \text{sign} \left(\sum_{m=1}^M a_m G_m(x) \right)$$

comprises credit card transactions made by European cardholders over two days in September 2013. It was collected as part of a collaborative project between the Université Libre de Bruxelles (ULB) and Worldline, a company specializing in money transfer services. The primary objective was to leverage big data techniques to identify fraudulent transactions. The dataset includes 30 features: the Time feature, representing the time elapsed since the first transaction; the Amount, indicating the transaction value; and 28 anonymized attributes labeled V1 to V28, obtained through Principal Component Analysis (PCA) to ensure the privacy and confidentiality of cardholders.

1) PERFORMANCE EVALUATION METRICS

To provide a comprehensive evaluation of the model's performance, the following metrics were employed:

- **Accuracy:** Measures the overall proportion of correctly classified instances:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (15)$$

- **Precision:** Indicates the proportion of correctly predicted positive instances among all predicted positives:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (16)$$

- **Recall (Sensitivity):** Reflects the model's ability to correctly identify all actual positive instances:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (17)$$

- **F1-Score:** Represents the harmonic mean of precision and recall:

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \quad (18)$$

- **Specificity:** Measures the ability to correctly identify negative instances:

$$\text{Specificity} = \frac{TN}{TN + FP} \quad (19)$$

- **AUC-ROC (Area Under the Curve):** Evaluates the classifier's ability to distinguish between classes by summarizing performance across all classification thresholds.

B. HYPERPARAMETERS AND ARCHITECTURE OF THE ADABOOST-LSTM SYSTEM

The base learner employed in the proposed ensemble is a deep Long Short-Term Memory (LSTM) neural network. Its architecture is specifically designed to handle time series or sequential patterns, even when reshaped from

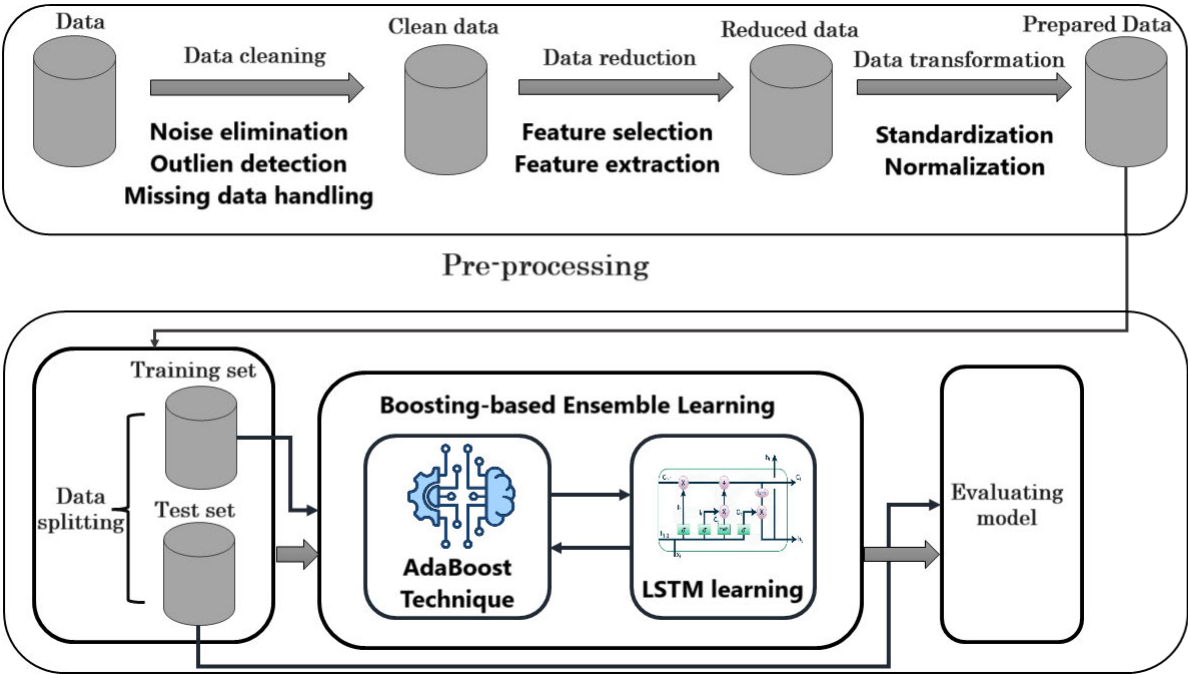


FIGURE 2. Architecture of the AdaBoost-LSTM hybrid framework.

TABLE 2. Features description.

Variable	Definition	Type
Class	Target feature indicating transaction legitimacy. Takes two values: 0 (Legitimate) and 1 (Fraud).	Categorical
Amount	Value of the transaction.	Numeric
Time	Time difference in seconds between the first and the current transaction.	Numeric
V1 to V28	PCA-transformed features for protecting cardholder privacy and confidentiality.	Numeric

tabular data. It comprises two stacked LSTM layers, each followed by a dropout layer to mitigate overfitting. The final dense layer uses a sigmoid activation function to perform binary classification. This architecture is detailed in Table 3. To ensure optimal performance and fair benchmarking, a set of hyperparameters was defined for both the LSTM model and the AdaBoost ensemble. These include the number of boosting iterations, learning rate, loss function, optimizer type, and data resampling strategy. The complete list of hyperparameters is provided in Table 4. These values were selected through preliminary tuning and are consistent with best practices in deep learning-based ensemble modeling.

TABLE 3. Architecture of the LSTM model used as base learner.

Layer	Configuration
Input	Input (shape=(1, number_of_features))
LSTM 1	64 units, return_sequences=True
Dropout 1	Rate = 0.2
LSTM 2	32 units
Dropout 2	Rate = 0.2
Dense (Output)	1 unit, activation = sigmoid

TABLE 4. Hyperparameters used in the AdaBoost-LSTM ensemble system.

Hyperparameter	Value / Description
Number of Boosting Iterations (T)	5, 10, 15
Loss Function	binary_crossentropy
Optimizer	Adam
Learning Rate	0.001
Batch Size	64
Number of Epochs	32 (per LSTM learner)
AdaBoost Weight Formula	$\alpha_m = \frac{1}{2} \log \left(\frac{1-\epsilon_m}{\epsilon_m} \right)$
Data Resampling Method	SMOTEENN
Cross-Validation	Stratified K-Fold $k = 5$)

C. RESULTS

The bar chart (Figure 4) illustrates the comparative performance of the AdaBoost-LSTM model with 5, 10, and 15 estimators across four key evaluation metrics—Accuracy, Precision, Recall, and F1-Score—while Figures 5 and 6 present the corresponding Precision-Recall and ROC curves, respectively. Together, these results highlight the model’s strong capability to distinguish between fraudulent and legitimate transactions under varying estimator configurations.

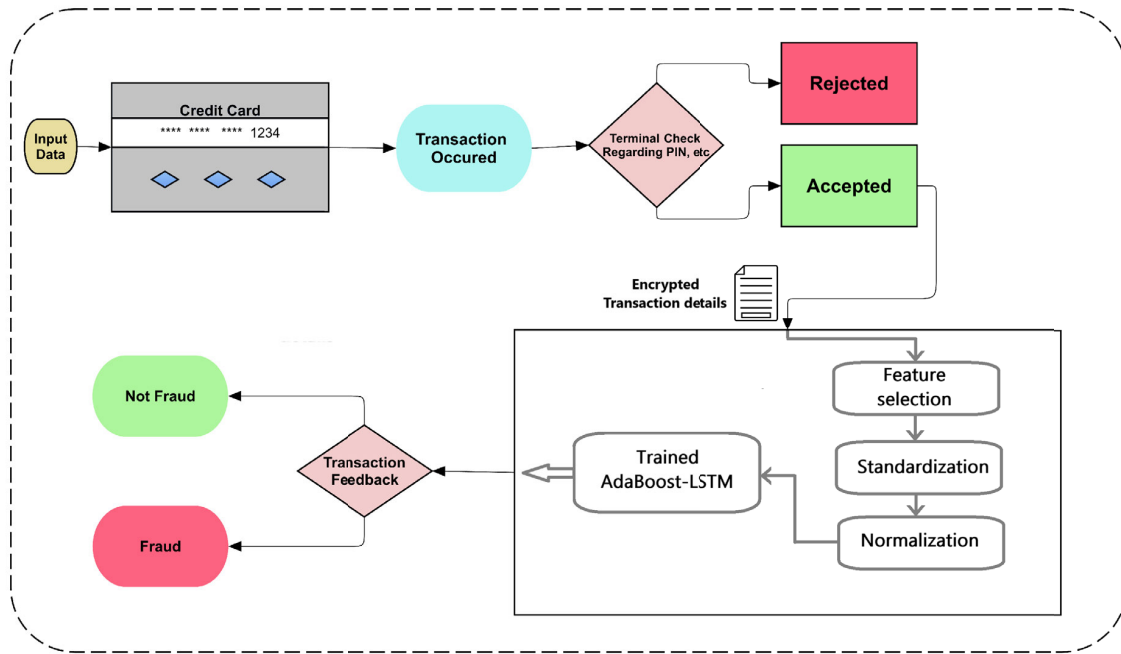


FIGURE 3. Credit card fraud detection approach.

Figure 4 reveals a consistent improvement across all evaluation metrics as the number of estimators increases. Accuracy rises from 0.9965 to 0.9990, indicating a notable enhancement in overall predictive performance. Precision follows a similar upward trend, reflecting the model’s improved capability to minimize false positives. Recall increases from 0.9972 to 0.9991, underscoring the model’s heightened sensitivity in correctly identifying positive instances. Consequently, the F1-Score—representing the harmonic mean of Precision and Recall—also improves, confirming the model’s balanced and robust classification performance.

This steady performance gain, with no observable signs of saturation, suggests that the AdaBoost-LSTM framework effectively benefits from the inclusion of additional estimators while avoiding overfitting. The performance enhancement can be attributed to the complementary strengths of AdaBoost’s adaptive weighting mechanism and LSTM’s ability to model temporal dependencies within sequential data. The near-perfect convergence of all metrics underscores the model’s reliability and stability across multiple evaluation criteria.

From a practical perspective, such consistent and high performance is particularly advantageous in domains where both false negatives and false positives carry substantial costs—such as medical diagnostics, fraud detection, and critical infrastructure monitoring. In these contexts, the strong balance between Recall and Precision achieved by the AdaBoost-LSTM model provides significant operational value.

Analysis of the Precision-Recall (Figure 5) and ROC curves (Figure 6) further validates the robustness and

consistency of the AdaBoost-LSTM framework. All configurations achieve near-perfect performance, with Average Precision (AP) and Area Under the Curve (AUC) values approaching 0.99. However, a slight performance difference is observed for the 5-estimator configuration, which exhibits marginally lower curve elevations compared to the 10- and 15-estimator models. This observation suggests that while all variants maintain exceptional discriminative power and stability, the 10-estimator configuration achieves the most balanced trade-off between model complexity and predictive accuracy.

Overall, these results demonstrate the AdaBoost-LSTM model’s remarkable capability to handle class imbalance and accurately identify fraudulent transactions. Its high precision and recall, coupled with stable generalization, make it highly suitable for real-world applications in finance, medical diagnostics, and critical infrastructure monitoring—where minimizing both false positives and false negatives is essential.

1) COMPARISON WITH OTHER ENSEMBLE LEARNING MODELS

The table 5 presents a comparative evaluation of several ensemble learning models—namely XGBClassifier, ExtraTreesClassifier, RandomForestClassifier, AdaBoost (classic), and the proposed AdaBoost-LSTM—based on key performance metrics, with all models trained and evaluated under an identical preprocessing pipeline to ensure fairness in comparison.

Based on the table 5, the proposed AdaBoost-LSTM model significantly outperforms traditional ensemble classifiers in

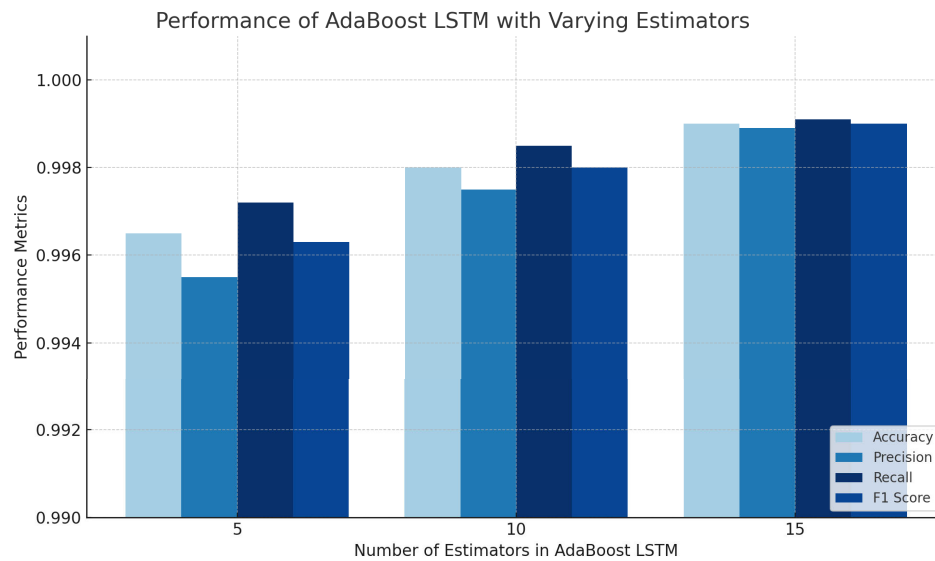


FIGURE 4. Performance of AdaBoost-LSTM with varying numbers of estimators.

TABLE 5. Performance of models in terms of accuracy, sensitivity, specificity, AUC, and average.

Model	Accuracy	Sensitivity	Specificity	AUC	Average
XGBClassifier	0.9985	0.8108	0.9870	0.8989	0.9238
ExtraTreesClassifier	0.9993	0.7703	0.9935	0.9150	0.9195
RandomForestClassifier	0.9993	0.7432	0.9950	0.9120	0.9124
AdaBoost (classic)	0.9985	0.8108	0.9870	0.8989	0.9238
AdaBoost LSTM	0.9990	0.9991	0.9989	0.9991	0.9990

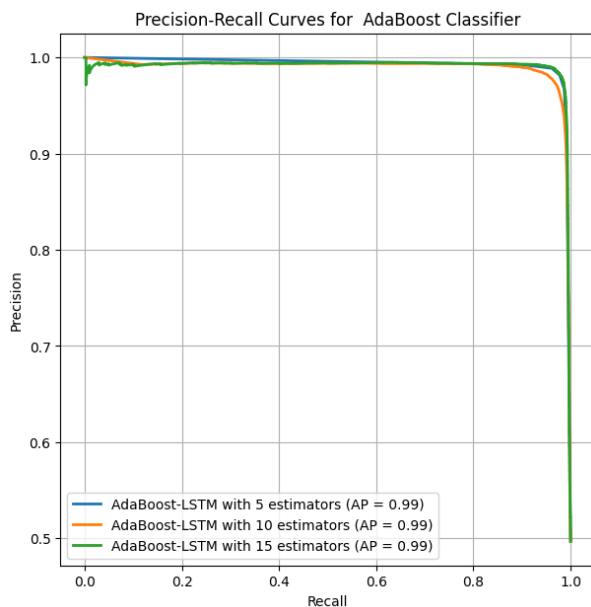


FIGURE 5. Precision-Recall curves for AdaBoost-LSTM classifier with different numbers of estimators.

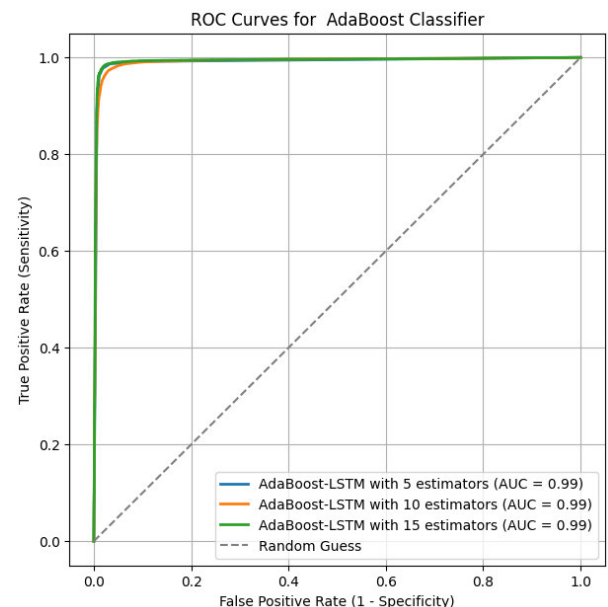


FIGURE 6. ROC curves for AdaBoost-LSTM classifier with different numbers of estimators.

the task of credit card fraud detection, achieving the highest scores across all evaluation metrics—particularly in Sensitivity (0.9991) and AUC (0.9991), which are crucial for identifying fraudulent transactions. Unlike conventional models

such as Random Forest, Extra Trees, and classic AdaBoost, which lack the ability to model temporal dependencies, the AdaBoost-LSTM leverages the sequential nature of transaction data through Long Short-Term Memory (LSTM) networks. By incorporating boosting techniques, the model

TABLE 6. Results credit card fraud detection.

Model	Accuracy	Sensitivity	Specificity	AUC	Average
ANN	0.9548	0.9827	0.9263	0.8636	0.9323
LSTM	0.9571	1	0.9252	0.8560	0.9641
RNN	0.9593	1	0.9293	0.8636	0.9752
ABDOST LSTM	0.9986	0.9986	0.9986	0.9987	0.9986

iteratively focuses on misclassified instances, while transfer learning between LSTM models enhances training efficiency and mitigates overfitting. This combination results in a robust classifier capable of capturing complex fraud patterns, with exceptional performance in terms of Accuracy (0.9990), Specificity (0.9989), and overall Average score (0.9990). These results demonstrate that integrating deep sequential learning into ensemble methods offers a powerful and effective framework for real-world fraud detection systems. Future work may explore its applicability in real-time environments, scalability, and adaptation to other domains involving sequential or multivariate data.

2) COMPARISONS WITH OTHER NEURAL NETWORKS

The table 6 presents a performance comparison between several neural network models—namely a basic Artificial Neural Network (ANN), Long Short-Term Memory (LSTM), Recurrent Neural Network (RNN), and the proposed AdaBoost-LSTM—on credit card fraud detection.

Based on the table 6 comparing different neural network models for credit card fraud detection, the AdaBoost-LSTM model clearly outperforms traditional neural networks such as ANN, LSTM, and vanilla RNN. While the classic neural networks show moderate accuracy (around 95-96%) and perfect sensitivity for LSTM and RNN, their specificity and AUC values are notably lower (around 0.85–0.93), indicating less reliable discrimination between fraudulent and legitimate transactions. In contrast, the AdaBoost-LSTM achieves near-perfect scores across all metrics, including Accuracy, Sensitivity, Specificity, and AUC (all approximately 0.9986), resulting in an outstanding average performance of 0.9986. This confirms the effectiveness of integrating boosting with sequential models, which not only captures temporal dependencies but also emphasizes hard-to-classify cases, leading to superior detection performance. The results suggest that AdaBoost-LSTM is a highly promising approach for robust and accurate fraud detection, significantly improving over traditional neural network architectures.

VI. CONCLUSION

This study introduces a novel hybrid framework for credit card fraud detection that effectively integrates the AdaBoost algorithm with Long Short-Term Memory (LSTM) networks, thereby leveraging both misclassification reweighting and temporal dependency modeling. The proposed AdaBoost-LSTM ensemble exhibits superior performance compared to conventional machine learning classifiers and standalone neural networks, achieving near-perfect results

in terms of accuracy, sensitivity, specificity, and AUC. By sequentially training LSTM models through an adaptive weighting mechanism and employing transfer learning to initialize each new model from the previous one, the framework significantly reduces computational overhead and mitigates overfitting. This design enables the ensemble to concentrate on hard-to-classify instances while capturing complex temporal dependencies inherent in transactional data.

Despite its promising performance, the proposed method is not without limitations. Although transfer learning alleviates computational costs, the framework's complexity may still pose challenges in large-scale or real-time deployment scenarios. Additionally, its dependence on labeled historical data constrains adaptability to emerging or previously unseen fraud patterns in dynamic environments.

Future research should focus on enhancing real-time fraud detection capabilities, reducing inference latency, and improving adaptability through online or continual learning approaches. Moreover, extending this framework to other domains characterized by sequential or multivariate data—such as cybersecurity, healthcare monitoring, and financial time-series forecasting—represents a promising direction for further investigation and practical application.

REFERENCES

- [1] K. AbdulSattar and M. Hammad, "Fraudulent transaction detection in FinTech using machine learning algorithms," in *Proc. Int. Conf. Innov. Intell. Informat., Comput. Technol. (3ICT)*, Dec. 2020, pp. 1–6.
- [2] I. Adamopoulos, A. Valamontes, J. T. Karantonis, P. P. Tropaitis, and N. Syrou, "Enhancing economic growth time series for UAE forecasting with deep learning: A Seq2Seq and attention-driven LSTM approach," *EDRAAK*, vol. 2025, pp. 62–75, Mar. 2025.
- [3] E. M. Al-dahasi, R. K. Alsheikh, F. A. Khan, and G. Jeon, "Optimizing fraud detection in financial transactions with machine learning and imbalance mitigation," *Expert Syst.*, vol. 42, no. 2, Feb. 2025, Art. no. e13682.
- [4] A. Alharbi, M. Alshammari, O. D. Okon, A. Alabrah, H. T. Rauf, H. Alyami, and T. Meraj, "A novel text2IMG mechanism of credit card fraud detection: A deep learning approach," *Electronics*, vol. 11, no. 5, p. 756, Mar. 2022.
- [5] A. Asselman, M. Khaldi, and S. Aammou, "Enhancing the prediction of student performance based on the machine learning XGBoost algorithm," *Interact. Learn. Environments*, vol. 31, no. 6, pp. 3360–3379, Aug. 2023.
- [6] R. B. Sulaiman, V. Schetinin, and P. Sant, "Review of machine learning approach on credit card fraud detection," *Human-Centric Intell. Syst.*, vol. 2, nos. 1–2, pp. 55–68, 2022.
- [7] G. D. C. Cavalcanti, J. P. Magalhaes, R. M. Barreto, and I. R. Tsang, "MLPBoost: A combined AdaBoost/multi-layer perceptron network approach for face detection," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2012, pp. 2350–2353.
- [8] S. Dhankhad, E. Mohammed, and B. Far, "Supervised machine learning algorithms for credit card fraudulent transaction detection: A comparative study," in *Proc. IEEE Int. Conf. Inf. Reuse Integr. (IRI)*, Jul. 2018, pp. 122–125.
- [9] J. Gupta, "Credit card fraud detection using machine learning algorithms," *Int. J. Sci. Res. (IJSR)*, vol. 12, no. 11, pp. 1774–1779, Nov. 2023.

- [10] O. E. Ejiofor, "A comprehensive framework for strengthening USA financial cybersecurity: Integrating machine learning and AI in fraud detection systems," *Eur. J. Comput. Sci. Inf. Technol.*, vol. 11, no. 6, pp. 62–83, Mar. 2023.
- [11] S. El Kafhali, M. Tayebi, and H. Sulimani, "An optimized deep learning approach for detecting fraudulent transactions," *Information*, vol. 15, no. 4, p. 227, Apr. 2024.
- [12] M. Frank, D. Drikakis, and V. Charissis, "Machine-learning methods for computational science and engineering," *Computation*, vol. 8, no. 1, p. 15, Mar. 2020.
- [13] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [14] H. Hu, Y. Wen, T.-S. Chua, and X. Li, "Toward scalable systems for big data analytics: A technology tutorial," *IEEE Access*, vol. 2, pp. 652–687, 2014.
- [15] F. Khan, S. Ateeq, M. Ali, and N. Butt, "Impact of COVID-19 on the drivers of cash-based online transactions and consumer behaviour: Evidence from a muslim market," *J. Islamic Marketing*, vol. 14, no. 3, pp. 714–734, Feb. 2023.
- [16] P. K. Sadineni, "Detection of fraudulent transactions in credit card using machine learning algorithms," in *Proc. 4th Int. Conf. I-SMAC (IoT Social, Mobile, Analytics Cloud) (I-SMAC)*, Oct. 2020, pp. 659–660.
- [17] X. Li, L. Wang, and E. Sung, "AdaBoost with SVM-based component classifiers," *Eng. Appl. Artif. Intell.*, vol. 21, no. 5, pp. 785–795, Aug. 2008.
- [18] H. Liu, H.-Q. Tian, Y.-F. Li, and L. Zhang, "Comparison of four AdaBoost algorithm based artificial neural networks in wind speed predictions," *Energy Convers. Manage.*, vol. 92, pp. 67–81, Mar. 2015.
- [19] V. C. M. Manduva, "Leveraging AI, ML, and DL for innovative business strategies: A comprehensive exploration," *Int. J. Modern Comput.*, vol. 5, no. 1, pp. 62–77, 2022.
- [20] A. George Jacob, "Fraudulent transactions detection using machine learning," *Int. J. Sci. Res. (IJSR)*, vol. 11, no. 9, pp. 463–469, Sep. 2022.
- [21] P. Raghavan and N. E. Gayar, "Fraud detection using machine learning and deep learning," in *Proc. Int. Conf. Comput. Intell. Knowl. Economy (ICCIKE)*, Dec. 2019, pp. 334–339.
- [22] A. Roy, J. Sun, R. Mahoney, L. Alonzi, S. Adams, and P. Beling, "Deep learning detecting fraud in credit card transactions," in *Proc. Syst. Inf. Eng. Design Symp. (SIEDS)*, Apr. 2018, pp. 129–134.
- [23] H. M. Saleh, A. K. Oleiwi, and A. A. H. Abed, "Detecting attacks in banks by cyber security: An applied study," *Babylonian J. Mach. Learn.*, vol. 2023, pp. 65–72, Nov. 2023.
- [24] N. Syrou, G. Mpourazanis, P. Tsirkas, A. Valamontes, and J. Adamopoulou, "Clinical data analysis using machine learning algorithms to predict the progression of type 2 diabetes," *Mesopotamian J. Artif. Intell. Healthcare*, vol. 2025, pp. 105–115, Jun. 2025.
- [25] A. Taherkhani, G. Cosma, and T. M. McGinnity, "AdaBoost-CNN: An adaptive boosting algorithm for convolutional neural networks to classify multi-class imbalanced datasets using transfer learning," *Neurocomputing*, vol. 404, pp. 351–366, Sep. 2020.
- [26] L. Wang, Z. Zhang, X. Zhang, X. Zhou, P. Wang, and Y. Zheng, "A deep-forest based approach for detecting fraudulent online transaction," *Adv. Comput.*, vol. 120, pp. 1–38, Nov. 2020.
- [27] W. Wang and D. Sun, "The improved AdaBoost algorithms for imbalanced data classification," *Inf. Sci.*, vol. 563, pp. 358–374, Jul. 2021.
- [28] C. Ying, Q.-G. Miao, J.-C. Liu, and G. Lin, "Advance and prospects of AdaBoost algorithm," *Acta Autom. Sinica*, vol. 39, no. 6, pp. 745–758, Mar. 2014.
- [29] Y. Zhang, X. Wang, J. Yu, T. Zeng, and J. Wang, "Adaboosting graph attention recurrent network: A deep learning framework for traffic speed forecasting in dynamic transportation networks with spatial-temporal dependencies," *Eng. Appl. Artif. Intell.*, vol. 127, Jan. 2024, Art. no. 107297.
- [30] J. Zheng, D. Lin, Z. Gao, S. Wang, M. He, and J. Fan, "Deep learning assisted efficient AdaBoost algorithm for breast cancer detection and early diagnosis," *IEEE Access*, vol. 8, pp. 96946–96954, 2020.



HAMZA H. SULIMANI received the B.S. degree in electrical and computer engineering from Umm Al-Qura University, Makkah, Saudi Arabia, in 2000, the M.S. degree in computer engineering from King Abdulaziz University, Jeddah, Saudi Arabia, in 2010, and the Ph.D. degree in information technology from the University of Technology Sydney (UTS), Australia, in 2023. Prior to pursuing the Ph.D. degree, he spent 17 years as a Senior Automation Engineer at Saudi Electricity Company, where he gained extensive experience in industrial systems and real-time control applications. His work aims to enhance the efficiency, scalability, and reliability of data-intensive applications in decentralized computing infrastructures. His current research interests include machine learning, distributed systems, and intelligent load balancing strategies, particularly within fog and edge computing environments.

...